# 설비 고장을 예측하는 AI 데이터 분석

## 예외처리 및 종합 실습

예외처리 기초

CONTENTS

# 목차


PART 1

## 오류와 예외의 이해

오류는 왜 생기는가 · 에러 메시지 읽기

PART 2

## try-except 구조

기본 구조 · 첫 실습

PART 3

## 구체적 예외 처리

자주 만나는 예외 · 여러 예외 동시 처리

# 오류는 왜 생기는가

**오류는 잘못의 신호가 아니라, 컴퓨터가 보내는 안내문입니다.**

중요한 건 오류를 **없애는 것**이 아니라, **다루는 것**입니다.

빨간 글씨는 잘못의 신호가 아니라, 컴퓨터가 **"여기서 시킨 일을 할 수가 없다"**라고 알려주는 안내문에 가깝습니다.

숙련된 개발자도 매일 오류를 만납니다. 차이는 **당황하지 않고 침착하게 대처하느냐**에 있습니다.

오늘 우리는 그 빨간 글씨를 정면으로 다뤄봅니다. 그 첫걸음은 오류 메시지에 익숙해지는 것입니다.

posco

# 오류가 생기는 세 가지 경우

오류는 발생 시점에 따라 세 종류로 나뉩니다. 오늘은 그중 실행 중 오류를 집중적으로 다룹니다.

CASE 01

## 문법 오류

괄호나 콜론을 빠뜨리거나
들여쓰기가 어긋난 경우

**실행되기 전에 발견됩니다.**

CASE 02 · 오늘의 핵심

## 실행 중 오류

0으로 나누기, 없는 파일 열기처럼
실행 도중 만나는 상황

**오늘 다룰 핵심 영역입니다.**

CASE 03

## 결과 오류

문법·실행은 다 되지만
결과가 틀린 경우

**가장 까다롭습니다.**

# 프로그램이 멈춘다는 것의 의미

**예외가 발생하면 그 줄에서 즉시 중단됩니다.**

그 아래 코드는 **한 줄도 실행되지 않습니다.**

프로그램은 위에서 아래로 한 줄씩 실행됩니다. 그러다 처리할 수 없는 상황을 만나면 그 자리에서 멈춥니다.

이것을 **"예외가 발생했다"**, 영어로 **Exception**이라고 부릅니다.

핵심은 이것입니다. 예외가 난 줄 아래 코드는 단 한 줄도 실행되지 않습니다. **30번째 줄에서 멈추면 31번째부터 100번째까지 전부 무시**됩니다.

# 단 한 줄이 전체를 멈춘다

센서 파일 50줄 중 단 한 줄의 불량 데이터가 전체 처리를 멈춥니다.

STEP 1

01

센서 파일 50줄을 위에서부터 한 줄씩 처리합니다.

STEP 2

02

10번째 줄에서 숫자 변환 실패.

STEP 3

그 줄에서 멈춰 나머지 40줄은 처리되지 못합니다.

posco

# 에러 메시지 읽는 법

**빨간 글씨 안에 문제를 푸는 모든 단서가 있습니다. 읽는 순서만 알면 됩니다.**

STEP 1 · 먼저 읽기

## 맨 아랫줄에서
## 오류 종류와 원인을 확인

콜론 앞은 **오류 종류**,
콜론 뒤는 **구체적 설명**입니다.

STEP 2 · 그 다음

## 그 위 line 줄에서
## 발생 위치 확인

**몇 번째 줄**에서 문제가 났는지
line 번호로 알려줍니다.

posco

# 트레이스백 읽는 순서

**맨 아래에 진짜 원인이 있습니다. 위로 갈수록 큰 줄기로 올라갑니다.**

```
Traceback (most recent call last):
File "sensor.py", line 10, in <module>
temp = int(value)
ValueError: invalid literal for int()
with base 10: '스물'
```

① 시작 알림
맨 윗줄 **Traceback...**은
읽지 않아도 됩니다.

② 그 다음 · 발생 위치
**line 10** — 10번째 줄을
살펴보면 됩니다.

③ 먼저 읽기 · 오류 종류·원인
맨 아랫줄에 **진짜 원인**이 있습니다.

# 실습 1. 트레이스백으로 에러 읽기

목표

**일부러 에러를 내고 트레이스백에서 오류 종류와 위치 찾기**

## 단계

① 글자가 섞인 값을 숫자로 바꾸려 시도해 오류를 일부러 발생시키기

② 어떤 수를 0으로 나누어 또 다른 오류를 발생시키기

③ 만들지 않은 변수를 사용해 세 번째 오류를 발생시키기

④ 각 오류마다 빨간 글씨의 맨 아랫줄에서 오류 이름을 확인

⑤ 그 윗줄에서 몇 번째 줄에서 났는지 위치를 확인

예상 결과 ValueError / ZeroDivisionError / NameError

posco

# 실습 1. 트레이스백으로 에러 읽기 정답

세 가지 오류를 일부러 내고, 트레이스백 맨 아랫줄에서 종류 확인

CODE · 정답

```
temp = int("스물")        # ValueError: 글자를 숫자로
result = 10 / 0           # ZeroDivisionError: 0으로 나눔
print(undefined_var)      # NameError: 정의 안 한 변수
```

# try-except 기본 구조

**try에 위험한 코드, except에 대처 방법을 적습니다.**

오류가 나도 프로그램은 **멈추지 않고 계속** 실행됩니다.

TRY

## 시도해봐

실패할 수도 있는 **위험한 코드**를 try 블록 안에 넣습니다.

EXCEPT

## 문제 생기면 이렇게

예외가 발생했을 때 어떻게 **대처할지**를 except 블록에 적습니다.

발상의 전환 : **"위험한 일이 안 일어나게 막는다"**가 아니라, **"일어나면 그때 대응한다"**입니다.

posco

# try-except 동작 코드

**오류가 나면 except로 점프합니다. 프로그램은 멈추지 않습니다.**

CODE

```
try:
temp = int("스물")
except:
print("못 바꿈")
```

▼ EXPLAIN

① try 안 **int("스물")**에서 오류 발생

② 즉시 아래 **except로 점프**

③ **"못 바꿈"** 출력 후 프로그램은 계속 실행

# try-except 사용 원칙

**잘 쓰는 세 가지 기준 : 들여쓰기 · 작게 · 다 잡지 않기**

01

## 들여쓰기

**콜론**으로 끝내고
안쪽 코드는 **한 단계 들여쓰기**

if문·for문과 동일한 규칙입니다.

02

## try는 작게

**위험한 코드만** 담아
오류 위치 파악에 유리하게

너무 많이 감싸면 어디서 문제 났는지 파악이 어려워집니다.

03

## 다 잡지 않기

모든 코드를 try로 감싸면
**진짜 버그까지 숨겨**

잘못된 결과를 그대로 믿게 됩니다.

# 실습 2. try-except로 오류 넘기기

**목표**

**try-except로 오류가 나도 프로그램이 멈추지 않게 만들기**

## 단계

① 온도를 입력값으로 받기

② 입력받은 온도를 숫자로 변환

③ 숫자로 변환에 성공했다면 온도를 출력

④ 만약 숫자로 변환이 불가능하다면 숫자를 입력하라는 안내문 출력

⑤ 어느 경우든 마지막에 프로그램이 계속된다는 문구를 출력

⑥ 정상값과 오류값을 각각 넣어 마지막 문구까지 실행되는지 확인

예상 결과  정상값 → 결과 / 오류값 → 안내 후 계속 진행

posco

# 실습 2. try-except로 오류 넘기기 정답

CODE · 정답

```
try:
    temp = int(input("온도: "))
    print("온도는", temp)
except ValueError:
    print("숫자를 입력하세요")
print("프로그램 계속")
```

CONSOLE

```
▸ 온도25
숫자를 입력하세요
프로그램 계속
```

**CHECKPOINT**

마지막 줄까지 실행되는 것이 핵심. 오류가 나도 프로그램은 계속 진행

posco

# 값이 잘못됐을 때 — ValueError

**타입은 맞지만 값이 부적절할 때 발생합니다.**

글자를 숫자로 바꿀 때 **가장 자주** 만나는 예외입니다.

EXAMPLE

```
int("62") # 정상
int("스물") # ValueError
```

▼ POINT

문자열이라는 **"종류"**는 맞습니다.
하지만 **"내용"**이 숫자로 바꿀 수 없는 값이라 거부됩니다.

**데이터 전처리에서 가장 많이 만나는 예외**입니다.
이 예외와 친해지는 것이 데이터 분석의 첫걸음입니다.

posco

# ValueError와 TypeError 구분

**"종류"가 틀린 것과 "내용"이 부적절한 것 — 비슷해 보이지만 다릅니다.**

01

## ValueError

종류는 맞고 **내용이 부적절**한 경우

```
int("abc")
int("3.5")
```

"3.5"도 ValueError입니다. int는 정수만 받습니다.

02

## TypeError

값의 **종류 자체가 틀린** 경우

```
10 + "개"
```

숫자와 문자열을 더하려 하면 발생합니다.

# 파일이 없을 때 — FileNotFoundError

**열려는 파일이 없을 때 발생합니다.**

**파일명 오타**와 **경로 문제**가 주요 원인입니다.

지난 시간 **open**으로 파일 읽는 법을 배웠습니다.

파일 이름을 **한 글자라도 틀리면** 이 FileNotFoundError를 만납니다.

이름 그대로 풀면 **"파일을 찾을 수 없음"** 오류입니다. open으로 열 때 그 파일이 그 위치에 없으면 발생합니다.

> **실무 포인트**
>
> 설비 데이터는 거의 항상 **CSV 파일을 열며 시작**하므로, 첫 단계에서 이 예외에 대비하는 것이 중요합니다.

posco

# FileNotFoundError 주요 원인

**대부분 세 가지 중 하나가 원인입니다.**

01

## 파일명 오타

**가장 흔한 원인**

sensor를 senser로 쓰거나
확장자 .csv를 빠뜨리는 경우

02

## 경로 문제

파일은 있는데 **다른 폴더**에 있는 경우

```
os.getcwd()
os.listdir()
```

03

## 확장자 숨김

윈도우에서 **.csv가 안 보일 수** 있습니다.
눈엔 sensor지만 실제로는 sensor.csv일 수 있습니다.

# 자주 만나는 그 밖의 에러

**예외 종류는 수십 가지지만, 초보 단계에서 만나는 건 손에 꼽힙니다.**

지금 다 외울 필요 없이, **만날 때마다 이름을 확인**하면 충분합니다.

프로그래밍은 **암기 과목이 아닙니다.**

큰 그림을 갖고, 빨간 글씨를 만나면 **맨 아랫줄을 읽어 종류를 확인**하면 됩니다.

두세 번 만나다 보면 자연스럽게 외워집니다.

> **접근 방식**
>
> 외우려 하지 말고 **맞닥뜨릴 때마다 정리**하는 습관을 만듭니다. 한 번 만난 예외는 다음에 훨씬 빨리 알아봅니다.

# 자주 만나는 예외 정리

**이 종류들만 알아도 초보 단계 예외의 대부분을 커버합니다.**

| 예외 | 발생 상황 | 예시 |
|---|---|---|
| `ValueError` | 값이 부적절 | `int("스물")` |
| `FileNotFoundError` | 파일이 없음 | `open` 실패 |
| `ZeroDivisionError` | 0으로 나눔 | 평균 계산 시 데이터 0개 |
| `IndexError` | 범위 초과 | 칸이 부족한 줄 |
| `KeyError` | 없는 키 접근 | 딕셔너리 검색 |
| `TypeError` | 값 종류가 틀림 | `10 + "개"` |

# 여러 예외 동시 처리

**예외마다 다른 대처가 필요하면 except를 여러 개 씁니다.**

파이썬은 **위에서부터 일치하는 첫 except**를 실행합니다.

실무 데이터는 **한 가지 오류만 나지 않습니다.**

어떤 줄은 글자가 섞여 **`ValueError`**, 어떤 줄은 칸 부족으로 **`IndexError`**가 날 수 있습니다.

각각 다른 안내를 주고 싶으면 except를 **여러 개 나란히** 적습니다.

> **실무 가치**
>
> 예외를 구분하면 **"글자 섞임 200건, 칸 부족 100건"**처럼 문제의 성격을 정확히 파악할 수 있습니다. 데이터 품질 관리와 직결됩니다.

posco

# 여러 except 코드

**발생한 예외 종류에 맞는 except가 골라져 실행됩니다.**

CODE

```
try:
    n = int(row[3])   # 칸이 2개뿐인 줄
except ValueError:
    print("숫자 아님")
except IndexError:
    print("칸 부족")
```

▼ EXPLAIN

글자를 int로 바꾸려 하면

**ValueError** 발생

해당 except가 골라져

**"숫자 아님"** 출력

없는 위치를 부르면

IndexError를 잡는 except가

실행

# 여러 예외 처리 원칙

**순서 · 구분 · 묶기 — 세 가지 기준으로 정리**

| 기준 | 내용 | 비고 |
|---|---|---|
| **순서** | 구체적 예외를 위에, 넓은 예외를 아래에 | 위에서부터 확인 |
| **구분** | 대처가 다르면 except를 나누어 작성 | 정확한 안내 가능 |
| **묶기** | 대처가 같으면 괄호로 묶어 처리 | `except (A, B):` |

# 실습 3. 구체적 예외로 입력 검증하기

목표

**구체적 예외를 지정해 잘못된 입력 검증하기**

## 단계

① 측정값을 입력값으로 받기

② 입력받은 값을 숫자로 변환

③ 100을 그 숫자로 나눈 결과를 출력

④ 만약 숫자로 변환이 불가능하다면 숫자가 아니라는 안내문 출력

⑤ 만약 입력값이 0이라면 0으로 나눌 수 없다는 안내문 출력

⑥ 두 안내문이 서로 다른 상황에서 나오는지 확인

예상 결과  잘못된 입력 → 안내 후 계속

# 실습 3. 구체적 예외로 입력 검증하기 정답

CODE · 정답

```
try:
    value = int(input("측정값: "))
    print(100 / value)
except ValueError:
    print("숫자가 아닙니다")
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다")
```

CONSOLE

```
▸ "덥다"
숫자 아님
```

CHECKPOINT

글자를 넣어도 **멈추지 않는지** 꼭 확인하세요.

# try-except만으로 부족한 세 가지

**오류를 받아넘기는 것만으로 해결되지 않는 세 가지 상황**

01

## 파일 닫기

오류 발생 시
그 아래 close 미실행

열어둔 파일이 그대로 남음

02

## 한 줄만 건너뛰기

반복 중 한 줄 실패로
전체 처리 중단

나머지 정상 줄까지 손실

03

## 정상으로 보이는 값

측정값 -999처럼
변환은 되는 값

예외 자체가 미발생